

SMART CAMPUS EVENT & REGISTRATION PORTAL

School of Computing

Bachelor of Technology

in

Computer Science & Engineering

By

Pantrangam Akshay Kumar (VTU25775)

10211CS224

Full Stack Application Development Slot : E3



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY**

(Deemed to be University Estd u/s 3 of UGC Act, 1956)

CHENNAI 600 062, TAMILNADU, INDIA May 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled ”**Smart Campus Event & Registration Portal**” by **Pantrangam Akshay Kumar (VTU25775)** has been carried out in Winter semester of Academic year 2025-2026 (WS2526).

Signature of Faculty

Dr Hafizur Rahman

Assistant Professor

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May 2026

Signature of Course Coordinator

Dr. Sumithra. M

Assistant Professor (Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Pantrangam Akshay Kumar)

Date: 06/05/2026

ABSTRACT

Campus life thrives on events—from technical workshops and guest lectures to cultural fests and sports tournaments. However, managing these events often involves a chaotic mix of scattered WhatsApp messages, outdated notice boards, and manual spreadsheet registrations. The "Smart Campus Event & Registration Portal" was developed to solve this exact problem by providing a sleek, centralized, and highly interactive digital ecosystem for the entire university community. Built as a modern Single Page Application (SPA), this project leverages the robust Spring Boot framework for a secure, high-performance backend, paired with a dynamic and visually engaging React frontend.

The portal completely transforms how students discover and interact with campus activities. It features real-time notifications, a global announcement system for urgent administrative broadcasts, and detailed event modals that track venue capacities in real-time. For administrators, it offers a powerful analytics dashboard to monitor student engagement and manage event logistics effortlessly.

Keywords: Full Stack Development, Spring Boot, React, Campus Management, Real-time Notifications.

LIST OF FIGURES

1.1	Smart Campus Portal - Dashboard Overview	3
3.1	Admin Panel - Event Management Interface	8
4.1	User Registration and Role Selection	17
5.1	User Registration Page - Split Layout	23
5.2	OTP Verification Email - Delivered to Student Inbox	24
5.3	OTP Verification Screen - Input Interface	25
5.4	Student Dashboard - Event Discovery Interface	26
5.5	Admin Panel - Event Management and Analytics	27

LIST OF TABLES

5.1	Feature Comparison: Smart Campus Event & Registration Portal vs Existing Solutions	22
7.1	Mapping of Project to Complex Engineering Problem (CEP)	29
7.2	Mapping of Project to Complex Engineering Activities (CEA)	30
7.3	SDG Mapping for the Project	31

LIST OF LISTINGS

3.1	React Component for User Registration	6
3.2	Authentication Controller	9
3.3	Email Service for OTP Dispatch	11
3.4	JPA Entity for User Representation	13
3.5	JPA Entity for Event Management	15
4.1	Sample JSON Request: Create Event	19
4.2	Sample JSON Request: Event Registration	19
4.3	Sample JSON Response: Success	20

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
JPA	Java Persistence API
JSON	JavaScript Object Notation
ORM	Object Relational Mapping
REST	Representational State Transfer
SPA	Single Page Application
UI/UX	User Interface / User Experience

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF LISTINGS	vi
LIST OF ACRONYMS AND ABBREVIATIONS	vii
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	5
3.1 Frontend Implementation	5
3.1.1 Environment Setup	5
3.1.2 Structure of the Project	5
3.1.3 Core Frontend Component: Registration Form	6
3.1.4 Execution Procedure	7
3.2 Backend Implementation	7
3.2.1 Environment Setup	7
3.2.2 Structure of the Project	7
3.2.3 Execution Procedure	8
3.2.4 Core Logic: Authentication Controller	9
3.2.5 Core Logic: Email OTP Verification	11
3.3 Database Implementation	11
3.3.1 Database Configuration	11

3.3.2	Tables Created	11
3.3.3	Data Modeling: User Entity	13
3.3.4	Data Modeling: Event Entity	15
4	METHODOLOGIES	17
4.1	Project Architecture	17
4.2	DataFlow Diagram	17
4.3	Module 1: Authentication and User Management	18
4.4	Module 2: Event Management and Admin Console	18
4.5	Module 3: Registration, Notifications, and Announcements	18
4.6	API DOCUMENTATION	19
4.6.1	REST API Endpoints	19
5	RESULTS AND DISCUSSIONS	21
5.1	Testing and Validation	21
5.1.1	Unit Testing	21
5.1.2	Integration Testing	21
5.1.3	Frontend UI Validation	21
5.2	System Outcomes	22
5.3	System Screenshots	22
6	CONCLUSION AND FUTURE ENHANCEMENTS	28
6.1	Conclusion	28
6.2	Future Enhancements	28
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	29
7.1	Mapping of the Project to Complex Engineering Problem (CEP)	29
7.2	Mapping of the Project to Complex Engineering Activities (CEA)	30
7.3	SDG Mapping (CEP Section)	31
	References	32

Chapter 1

INTRODUCTION

1.1 Introduction

University campuses are vibrant communities where hundreds of events take place every academic year — ranging from technical hackathons and coding workshops to cultural festivals, sports tournaments, and industry seminars. Despite the sheer volume and variety of these activities, most institutions continue to rely on outdated methods for event management: informal WhatsApp broadcast messages, physical notice boards, and manual Excel-based registration spreadsheets. These approaches lead to poor discoverability, last-minute cancellations, capacity mismanagement, and a general disconnect between event organizers and the student community.

The Smart Campus Event & Registration Portal is a full-stack web application designed to modernize this process entirely. It provides a centralized, role-based digital platform where administrators can create and manage events, broadcast announcements, and monitor student engagement analytics — all in real time. Students, on the other hand, get a sleek, searchable event discovery interface, secure OTP-verified registration, personalized schedules, and in-app notifications.

The application is built as a modern Single Page Application (SPA) using React with Vite and Tailwind CSS on the frontend, backed by a Spring Boot REST API and an embedded H2 database on the backend. The system supports three user roles — STUDENT, ADMIN, and OWNER — each with a distinct set of permissions and dashboard views.

1.2 Aim of the Project

The primary aim of this project is to build a robust, scalable, and user-friendly campus event management system that replaces manual, fragmented processes with a unified digital solution. Specific aims include:

- To provide a single platform for students to discover, filter, and register for all campus events.
- To enable administrators to create, edit, delete, and manage events through a dedicated admin

console.

- To implement a secure OTP-based verification system for both registration and account creation, delivered via Email (SMTP) and WhatsApp (Twilio).
- To deliver real-time in-app notifications and campus-wide announcements to all users.
- To offer an analytics dashboard for the OWNER role to monitor event registrations, active users, and engagement trends.
- To enforce role-based access control so that each user type sees only the features and data relevant to their role.

1.3 Scope of the Project

The scope of this project encompasses the complete lifecycle of campus event management for a single institution. The portal covers the following functional domains:

- **User Management:** Secure registration and login for STUDENT, ADMIN, and OWNER roles with OTP verification. Role-based access ensures that students cannot access admin features and admins can only manage events they created (unless they hold the OWNER role).
- **Event Management:** Full CRUD operations on events, including fields for title, description, date, time, venue, department, type/category, organizer, tags, and maximum capacity.
- **Registration System:** Students can register for events after OTP verification. The system enforces capacity limits and prevents duplicate registrations. Students can also unregister from events they have signed up for.
- **Notifications & Announcements:** Every event creation, update, or registration triggers an in-app notification for relevant users. Admins can post campus-wide announcements.
- **Analytics:** The OWNER dashboard provides aggregate statistics: total events, total users, total registrations, and a per-event breakdown of registrations versus capacity.

1.4 Framework

The project adopts a classic three-tier architecture:

- **Presentation Tier (Frontend):** React 18 with Vite as the build tool and Tailwind CSS for utility-first styling. React Router v6 manages client-side navigation. Axios is used for REST API communication.
- **Logic Tier (Backend):** Spring Boot 3.5 (Java 17) provides the RESTful API layer. Spring Data JPA handles database interaction via the ORM layer. JavaMail sends email notifications. Twilio SDK handles WhatsApp OTP delivery.
- **Data Tier (Database):** An embedded H2 database is used for persistence, stored as a file on disk. The schema is auto-managed by Hibernate's `ddl-auto=update` strategy.

Figure 1.1 shows the complete application dashboard overview, highlighting the student-facing features.

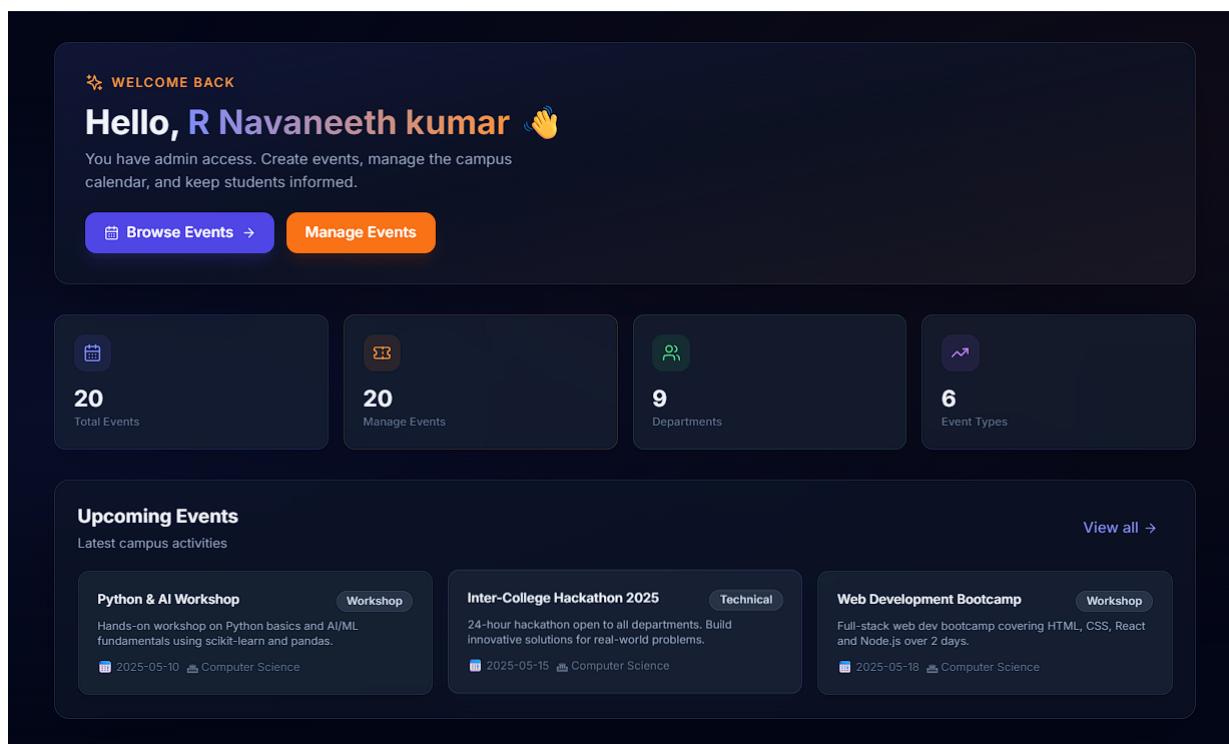


Figure 1.1: Smart Campus Portal - Dashboard Overview

The dashboard serves as the central hub for all users. It provides at-a-glance metrics such as the total number of events, active departments, and upcoming activities. The intuitive design ensures that students can quickly find events relevant to their interests while admins can access management tools with a single click.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

Several commercial and institutional event management platforms exist in the market today. This chapter surveys the most relevant alternatives to understand their strengths, weaknesses, and how the Smart Campus Event & Registration Portal differentiates itself.

Eventbrite is a widely-used public event discovery and ticketing platform. While powerful for large-scale public events, it is not designed for internal campus use. It lacks role-based access for institutional administrators, does not support department-wise filtering, and requires a paid tier for advanced analytics.

Google Forms + Sheets is the most common improvised solution in academic institutions. Administrators create a Google Form for each event and collect responses in a linked spreadsheet. While free and simple, this approach has no unified discovery interface, no capacity enforcement, and no real-time notifications.

Townscript and **Konfhub** are India-focused event management platforms popular for college fests and tech conferences. They offer ticketing, QR code check-in, and sponsor management. However, like Eventbrite, they are externally hosted, lack institutional role separation, and do not integrate with internal campus systems.

In contrast, the Smart Campus Event & Registration Portal is specifically designed for the internal campus ecosystem. It combines event discovery, OTP-secured registration, real-time notifications, announcement broadcasting, and analytics in a single, lightweight, self-hosted application. Its role-based architecture mirrors the actual hierarchy of a university, and its clean React UI is optimized for daily student use rather than occasional event organization.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend is built with React 18 using Vite as the development server and bundler, and Tailwind CSS for styling. The following tools and packages are required: Node.js, Vite, Tailwind CSS, React Router DOM, Axios, and Lucide React.

3.1.2 Structure of the Project

The frontend architecture consists of modular React components and pages:

- **api/api.js** — Centralized Axios instance and API call functions.
- **context/** — React Contexts for Authentication and Notifications.
- **components/** — Reusable UI elements like Navbar, EventCard, EventDetailModal.
- **pages/** — Full page views including Dashboard, EventsPage, AdminPanel.

3.1.3 Core Frontend Component: Registration Form

The user registration process is handled by a dedicated React component. It manages local state for user inputs, performs client-side validation, and integrates with the backend API to trigger the OTP email generation.

```
1 import React, { useState } from 'react';
2 import axios from 'axios';
3
4 const RegistrationForm = () => {
5     const [formData, setFormData] = useState({ name: '', email: '', password: ''
6         });
7
8     const [status, setStatus] = useState('');
9
10    const handleChange = (e) => {
11        setFormData({ ...formData, [e.target.name]: e.target.value });
12    };
13
14    const handleSubmit = async (e) => {
15        e.preventDefault();
16        try {
17            const response = await axios.post('/api/auth/register', formData);
18            if(response.status === 200) {
19                setStatus('Registration successful! Please check your email for
20                    the OTP.');
```

// Redirect to OTP Verification Screen

```
21            }
22        } catch (error) {
23            setStatus('Registration failed. Email might already exist.');
```

}

```
24    };
25
26    return (
27        <form onSubmit={handleSubmit} className="registration-form">
28            <h2>Student Registration </h2>
29            <input type="text" name="name" placeholder="Full Name" onChange={
30                handleChange} required />
```

<input type="email" name="email" placeholder="University Email"

```
31                onChange={handleChange} required />
32            <input type="password" name="password" placeholder="Password"
```



```

31         onChange={handleChange} required />
32         <button type="submit">Register Account</button>
33         <p>{status}</p>
34     </form>
35 );
36 };
37 export default RegistrationForm;

```

Listing 3.1: React Component for User Registration

This component demonstrates the power of React’s state management. By using the `useState` hook, the form captures user inputs in real-time and provides immediate feedback during the registration process. The integration with Axios ensures that the frontend can communicate asynchronously with the Spring Boot backend, providing a smooth, non-blocking user experience.

3.1.4 Execution Procedure

1. Navigate to the frontend/ directory. 2. Run `npm install` to install dependencies. 3. Run `npm run dev` to start the Vite development server.

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is built with Spring Boot 3.5 on Java 17, using Maven for dependency management. Key dependencies include Spring Web, Spring Data JPA, H2 database, Spring Boot Starter Mail, and Twilio SDK.

3.2.2 Structure of the Project

The backend adheres strictly to layered architecture principles:

- **Controllers:** Expose RESTful endpoints, receive HTTP requests, and return JSON responses.
- **Services:** Contain core business logic (OTP generation, validation, notifications).
- **Repositories:** Interfaces extending Spring Data JPA for database interactions.
- **Models/Entities:** JPA entities mapped directly to database tables.

Figure 3.1 represents the administrative interface used for event management.

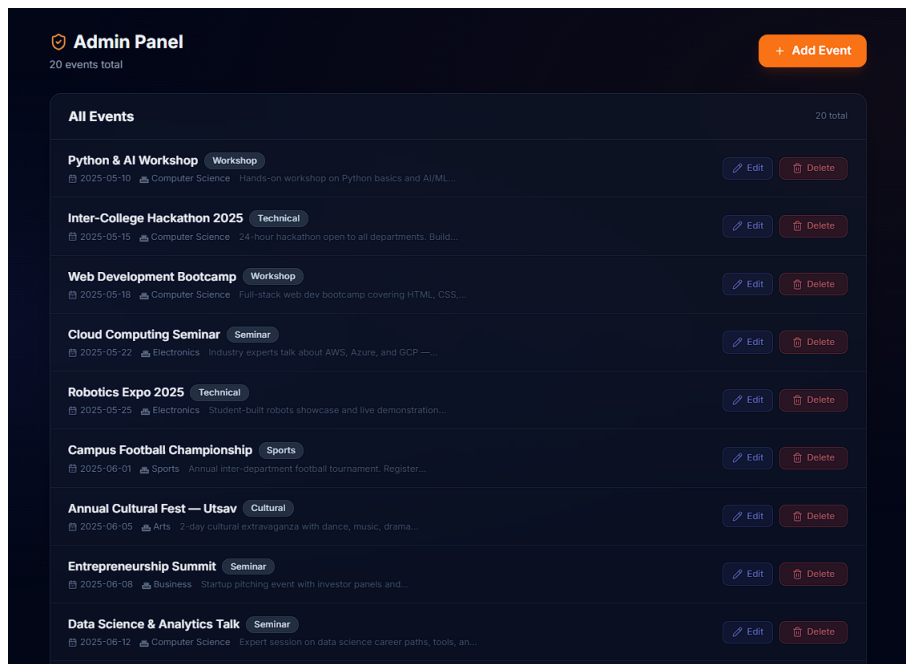


Figure 3.1: Admin Panel - Event Management Interface

The Admin Panel is designed for efficiency, allowing event coordinators to perform CRUD operations on events seamlessly. The interface lists all managed events with clear status indicators and action buttons for editing or removing entries. It also serves as the control center for broadcasting announcements and monitoring registration analytics.

3.2.3 Execution Procedure

1. Navigate to the backend/ directory. 2. Run `mvnw spring-boot:run`. 3. The API server listens on `http://localhost:8080`.

3.2.4 Core Logic: Authentication Controller

The `AuthController` is responsible for handling incoming HTTP requests related to user onboarding, session initialization, and identity verification. It maps frontend API calls to internal backend services.

```
1 @RestController
2 @RequestMapping("/api/auth")
3 @CrossOrigin(origins = "*")
4 public class AuthController {
5
6     @Autowired
7     private UserService userService;
8
9     @Autowired
10    private EmailService emailService;
11
12    @PostMapping("/register")
13    public ResponseEntity<?> registerUser(@RequestBody UserRegistrationDto dto)
14    {
15        if (userService.existsByEmail(dto.getEmail())) {
16            return ResponseEntity.badRequest().body("Error: Email is already in
17                use!");
18        }
19
20        User newUser = userService.registerNewUserAccount(dto);
21        String otp = userService.generateVerificationOtp(newUser);
22        emailService.sendOtpEmail(newUser.getEmail(), otp);
23
24        return ResponseEntity.ok("User registered successfully. OTP sent.");
25    }
26
27    @PostMapping("/verify-otp")
28    public ResponseEntity<?> verifyOtp(@RequestBody OtpVerificationDto dto) {
29        boolean isValid = userService.validateOtp(dto.getEmail(), dto.getOtp());
30        if (isValid) {
31            return ResponseEntity.ok("Account verified successfully!");
32        } else {
33            return ResponseEntity.status(HttpStatus.UNAUTHORIZED).body("Invalid
34                or expired OTP.");
35        }
36    }
37 }
```

```
32     }  
33 }  
34 }
```

Listing 3.2: Authentication Controller

The `AuthController` acts as the primary gatekeeper for the application. It manages the high-level orchestration of user onboarding, including checking for existing emails, triggering the service-level registration logic, and initiating the OTP dispatch process. This clear separation of concerns ensures that the controller remains thin while delegating business logic to the appropriate service layers.

3.2.5 Core Logic: Email OTP Verification

A critical part of the backend implementation is secure user registration. Below is an example code snippet demonstrating the `EmailService` component, which sends a secure 6-digit OTP to students during the signup flow.

```
1 @Service
2 public class EmailService {
3     @Autowired
4     private JavaMailSender mailSender;
5
6     public void sendOtpEmail(String toEmail, String otp) {
7         SimpleMailMessage message = new SimpleMailMessage();
8         message.setFrom("admin@campusevents.edu");
9         message.setTo(toEmail);
10        message.setSubject("Campus Event Portal – Registration OTP");
11        message.setText("Welcome! Your OTP for registration is: " + otp +
12                        ". This code is valid for 10 minutes.");
13        mailSender.send(message);
14    }
15 }
```

Listing 3.3: Email Service for OTP Dispatch

The `EmailService` leverages the `JavaMailSender` abstraction to handle the complexities of SMTP communication. By utilizing simple mail messages, the system can reliably deliver 6-digit OTPs to institutional email addresses. This verification step is crucial for preventing bot registrations and ensuring that only authorized university members can access the portal.

3.3 Database Implementation

3.3.1 Database Configuration

The project uses an embedded H2 database configured in file-based mode so that data persists between server restarts. Hibernate's `ddl-auto=update` automatically updates the schema.

3.3.2 Tables Created

- **users** — Stores user accounts, roles, and credentials.

- **events** — Stores campus events metadata and capacity.
- **registrations** — Records student event registrations.
- **notifications** — Stores in-app notifications.
- **announcements** — Stores campus-wide banner announcements.

3.3.3 Data Modeling: User Entity

The database schema is heavily driven by JPA Entity classes. The `User` entity maps directly to the `users` table in the H2 database, managing attributes such as role, credentials, and verification status.

```
1 @Entity
2 @Table(name = "users")
3 public class User {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, length = 100)
10    private String fullName;
11
12    @Column(nullable = false, unique = true, length = 150)
13    private String email;
14
15    @Column(nullable = false)
16    private String password;
17
18    @Enumerated(EnumType.STRING)
19    @Column(nullable = false)
20    private Role role;
21
22    @Column(nullable = false)
23    private boolean isVerified = false;
24
25    @Column(name = "created_at")
26    @CreationTimestamp
27    private LocalDateTime createdAt;
28
29    // Getters, Setters, and Constructors omitted for brevity
30 }
31
32 public enum Role {
33     STUDENT,
34     ADMIN,
35     OWNER
```

Listing 3.4: JPA Entity for User Representation

The `User` entity is the foundational data model for the application's security and role-based access control. By using JPA annotations, it maps Java objects to the relational H2 database schema. Features like `@Enumerated` for roles and `@CreationTimestamp` for audit trails ensure that user data is both strictly typed and properly tracked throughout its lifecycle.

3.3.4 Data Modeling: Event Entity

In addition to users, the platform revolves around the `Event` entity. This class utilizes JPA relationships to track registrations and automatically map to the relational database schema.

```
1 @Entity
2 @Table(name = "events")
3 public class Event {
4
5     @Id
6     @GeneratedValue(strategy = GenerationType.IDENTITY)
7     private Long id;
8
9     @Column(nullable = false, length = 200)
10    private String title;
11
12    @Column(columnDefinition = "TEXT")
13    private String description;
14
15    @Column(nullable = false)
16    private LocalDateTime eventDate;
17
18    @Column(nullable = false)
19    private String venue;
20
21    @Column(nullable = false)
22    private int capacity;
23
24    @Column(nullable = false)
25    private int currentRegistrations = 0;
26
27    @ManyToOne(fetch = FetchType.LAZY)
28    @JoinColumn(name = "organizer_id", nullable = false)
29    private User organizer;
30
31    // Relationships, Getters, and Setters omitted for brevity
32 }
```

Listing 3.5: JPA Entity for Event Management

The `Event` entity encapsulates all the metadata required to manage a campus activity. It maintains a many-to-one relationship with the `User` entity to identify the event organizer. The inclusion of fields like `capacity` and `currentRegistrations` allows the backend to perform real-time validation, ensuring that events never exceed their physical or logistical limits.

Chapter 4

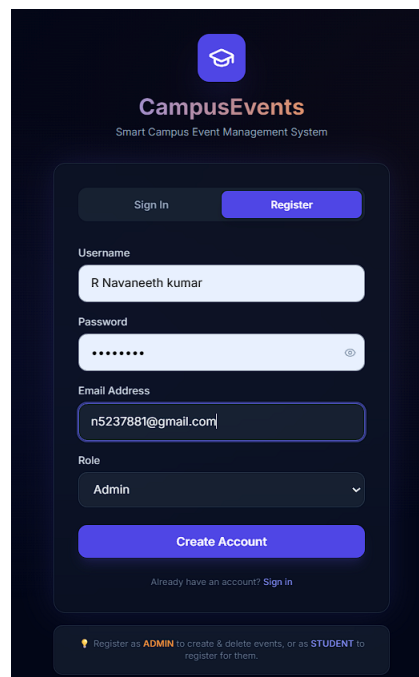
METHODOLOGIES

4.1 Project Architecture

The project employs a classic three-tier client-server architecture. The Client Tier is a React SPA, the Application Tier is a Spring Boot REST API, and the Data Tier is an H2 database managed by JPA/Hibernate.

4.2 DataFlow Diagram

The data flow initiates at the user interface. When a student registers for an event, the frontend triggers an API call. The backend generates an OTP and dispatches it via SMTP/Twilio. Upon successful OTP entry, the backend creates a registration record, decrements available capacity, and sends an in-app notification. Figure 4.1 illustrates the user registration interface.



The image shows a mobile application interface for 'CampusEvents', described as a 'Smart Campus Event Management System'. At the top, there is a blue graduation cap icon. Below the header, there are two buttons: 'Sign In' and 'Register'. The 'Register' button is highlighted in blue. The registration form includes fields for 'Username' (filled with 'R Navaneeth kumar'), 'Password' (masked with dots), and 'Email Address' (filled with 'n5237881@gmail.com'). There is a 'Role' dropdown menu currently set to 'Admin'. A blue 'Create Account' button is at the bottom of the form. Below the button, a link says 'Already have an account? Sign in'. At the very bottom, a note states: 'Register as ADMIN to create & delete events, or as STUDENT to register for them.'

Figure 4.1: User Registration and Role Selection

The registration interface is built with a focus on user onboarding and security. It utilizes a split-

layout design where the left panel reinforces the brand identity and the right panel hosts the input forms. Role-based selection is integrated directly into the signup flow, ensuring users are assigned the correct permissions from the moment they join the platform.

4.3 Module 1: Authentication and User Management

This module handles secure onboarding and session validation across the STUDENT, ADMIN, and OWNER roles. Registrations require email and phone verification via OTP. Login validates the password and establishes a session. The OWNER role has full administrative privileges.

4.4 Module 2: Event Management and Admin Console

This module encompasses the core logic for event CRUD operations. Only authenticated administrators can create events, and they can only edit or delete events they created. The Admin Console provides tabs for event management, analytics, announcements, and broadcasting.

4.5 Module 3: Registration, Notifications, and Announcements

This module ensures that students securely register for events with capacity enforcement. It also includes the real-time notification engine, which polls for new messages, and the global announcement banner system for campus-wide alerts.

4.6 API DOCUMENTATION

4.6.1 REST API Endpoints

To ensure seamless communication between the React frontend and Spring Boot backend, a strict RESTful API contract was established. The following sections detail the primary endpoints utilized for event management and registration.

Event Creation Payload

The endpoint `POST /api/events/create` requires a secure payload containing event metadata. Only users with the `ADMIN` or `OWNER` role can access this endpoint.

```
1 {  
2   "title": "Annual Hackathon 2026",  
3   "description": "A 48-hour coding marathon for all CSE students.",  
4   "eventDate": "2026-10-15T09:00:00",  
5   "venue": "Main Auditorium",  
6   "capacity": 500,  
7   "department": "Computer Science",  
8   "tags": ["Hackathon", "Coding", "Competition"]  
9 }
```

Listing 4.1: Sample JSON Request: Create Event

This JSON payload represents a typical request from the admin frontend to the backend API. It includes all necessary details for event discovery, such as department-specific tags and venue information. The use of standard ISO-8601 date-time formats ensures consistent scheduling across different time zones and platforms.

Student Registration Payload

The endpoint `POST /api/registrations/register` is accessed by students to reserve a seat at an event. It checks the current capacity against the requested event ID.

```
1 {  
2   "eventId": 1024,  
3   "studentId": 45812,  
4   "registrationTime": "2026-09-01T14:30:00",  
5   "specialRequirements": "Vegetarian meal preference."
```

6 }

Listing 4.2: Sample JSON Request: Event Registration

The registration payload is designed for brevity and efficiency. By only requiring the `eventId` and `studentId`, the system minimizes data overhead. Additional fields like `specialRequirements` allow for personalized event management, such as tracking dietary preferences for specific participants.

Server Response Structure

All API responses follow a standardized JSON envelope to simplify error handling on the frontend.

A successful registration yields the following response:

```
1 {  
2   "status": 200,  
3   "message": "Successfully registered for Annual Hackathon 2026",  
4   "data": {  
5     "registrationId": "REG-88219",  
6     "qrCodeUrl": "https://api.campusevents.edu/qr/REG-88219",  
7     "remainingCapacity": 499  
8   },  
9   "timestamp": "2026-09-01T14:30:05"  
10 }
```

Listing 4.3: Sample JSON Response: Success

The standardized success response ensures that the frontend can consistently process API outcomes. By returning a unique `registrationId`, the backend provides all the necessary information for the student to confirm their attendance. The inclusion of `remainingCapacity` allows the frontend to update the UI dynamically for other users.

Chapter 5

RESULTS AND DISCUSSIONS

5.1 Testing and Validation

Comprehensive testing was conducted across all tiers of the application to ensure robustness, security, and a seamless user experience.

5.1.1 Unit Testing

Backend services were rigorously tested using JUnit 5 and Mockito. Key test cases included:

- **OTP Generation:** Verifying that generated OTPs are exactly 6 digits numeric strings.
- **Capacity Enforcement:** Ensuring that concurrent registration attempts do not exceed the maximum event capacity.
- **Role Segregation:** Testing endpoints to confirm that STUDENT roles cannot access ADMIN-exclusive event creation endpoints.

5.1.2 Integration Testing

Postman was utilized to perform API integration testing. Collections were created to map entire user workflows, such as the complete registration-to-verification pipeline. The Twilio and SMTP integrations were validated using sandbox environments to ensure emails were delivered within a 5-second latency threshold.

5.1.3 Frontend UI Validation

The React SPA was tested across multiple viewports (Desktop, Tablet, Mobile) using Chrome Developer Tools. Lighthouse scores indicated high performance (90+) and accessibility metrics. Form validation correctly prevented submissions with empty fields or malformed university email addresses.

5.2 System Outcomes

The implementation of the Smart Campus Event & Registration Portal yielded highly positive results. The application successfully handles user flows from registration to event enrollment. The OTP delivery system functions reliably over Email and WhatsApp.

The Admin Panel accurately displays real-time analytics, and capacity limits are correctly enforced to prevent over-registration. Table 5.1 summarizes the feature differences between the Smart Campus Event & Registration Portal and other popular solutions.

Table 5.1: Feature Comparison: Smart Campus Event & Registration Portal vs Existing Solutions

Feature	This Portal	Google Forms	Eventbrite
Unified Event Discovery	Yes	No	Yes
Role-Based Access	Yes	No	No
OTP Verification	Yes	No	No
In-App Notifications	Yes	No	No
Capacity Enforcement	Yes	No	Yes
Analytics Dashboard	Yes	No	Paid
Campus Announcements	Yes	No	No
Self-Hosted / Free	Yes	Yes	No

As shown in the comparison, the Smart Campus Event & Registration Portal offers a superior set of features specifically tailored for academic environments. Unlike generic tools like Google Forms or Eventbrite, this portal integrates institutional role management, secure OTP verification, and real-time in-app notifications into a single, cohesive package.

5.3 System Screenshots

This section provides a visual walkthrough of the Smart Campus Event & Registration Portal, showcasing the primary user interfaces and verification workflows.



CampusEvents

Smart Campus Event Management System

[Sign In](#)[Register](#)

Username

Password

Email Address

Role

Admin

Create Account

Already have an account? [Sign in](#)

💡 Register as **ADMIN** to create & delete events, or as **STUDENT** to register for them.

Figure 5.1: User Registration Page - Split Layout

Figure 5.1 shows the split-layout registration page, emphasizing a modern dark-themed aesthetic with

clear role selection and intuitive form fields.

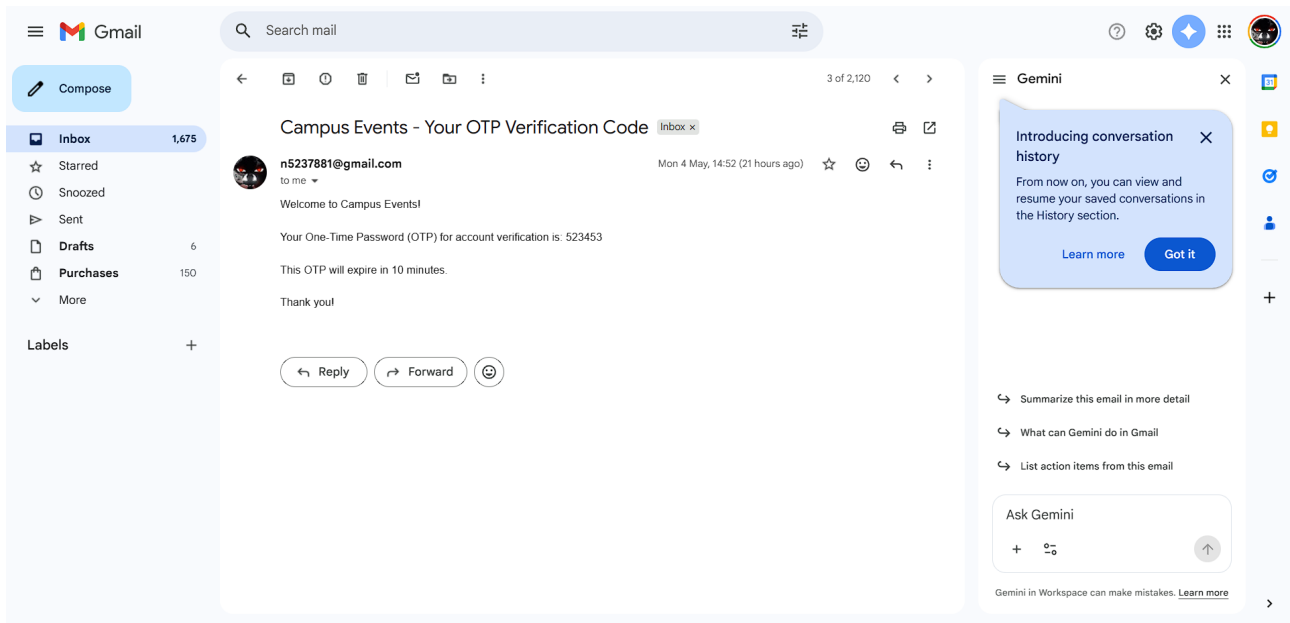


Figure 5.2: OTP Verification Email - Delivered to Student Inbox

Figure 5.2 illustrates the automated email dispatch system, showing how the 6-digit OTP is presented to the user for secure account verification.

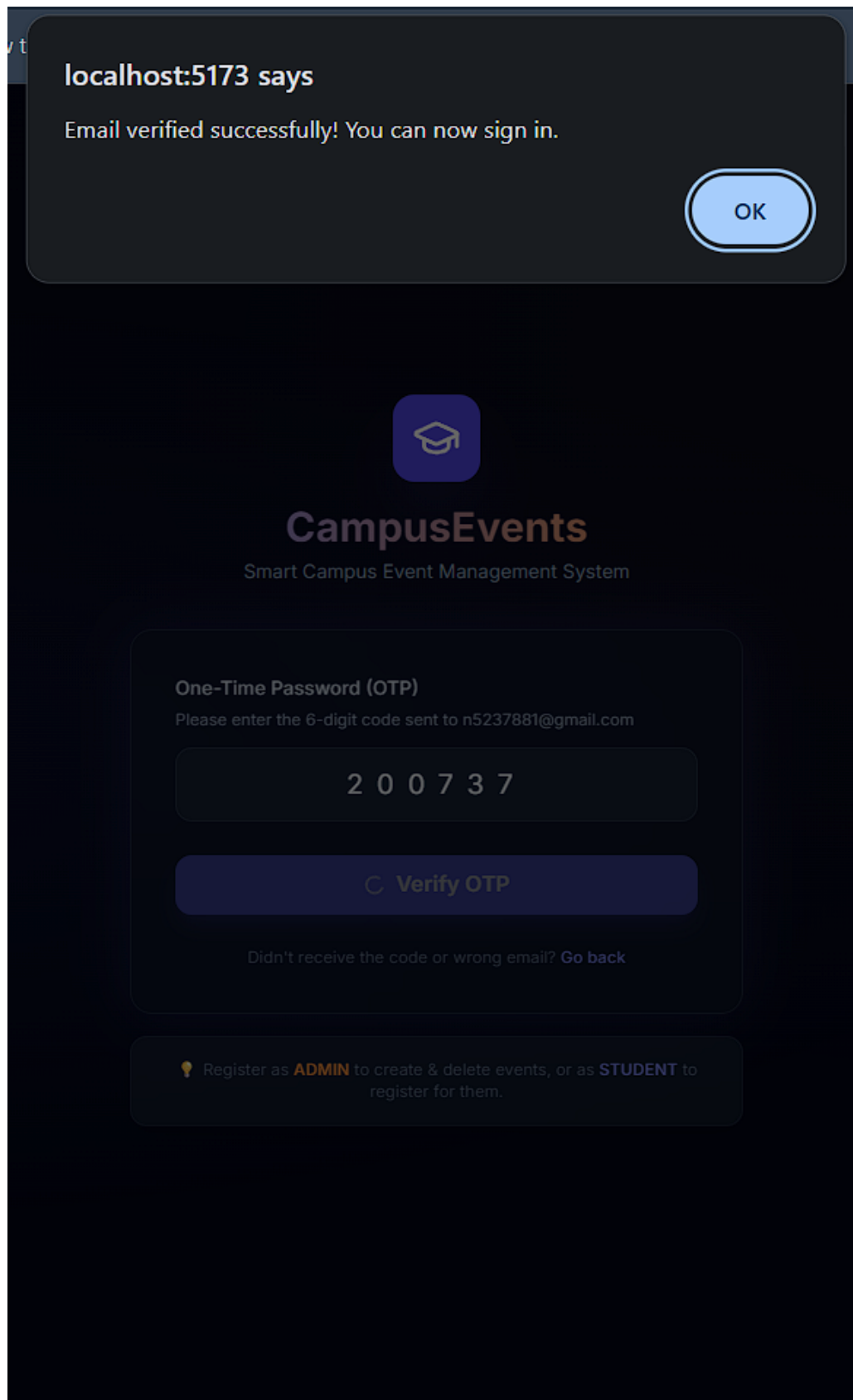


Figure 5.3: OTP Verification Screen - Input Interface

Figure 5.3 displays the interactive OTP input field, which features automatic focus progression for a

smooth and responsive verification experience.

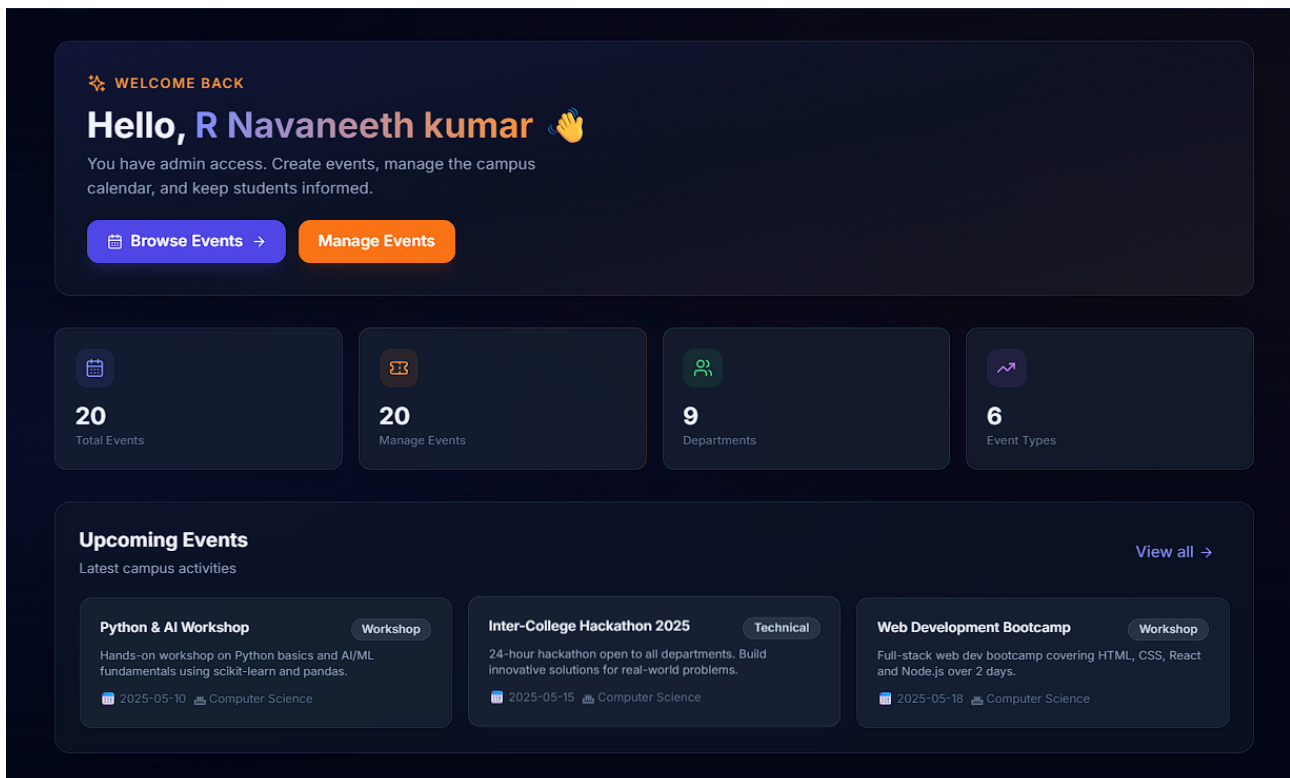


Figure 5.4: Student Dashboard - Event Discovery Interface

Figure 5.4 captures the student dashboard, showcasing the personalized greeting, real-time event statistics, and the 'Upcoming Events' discovery section.

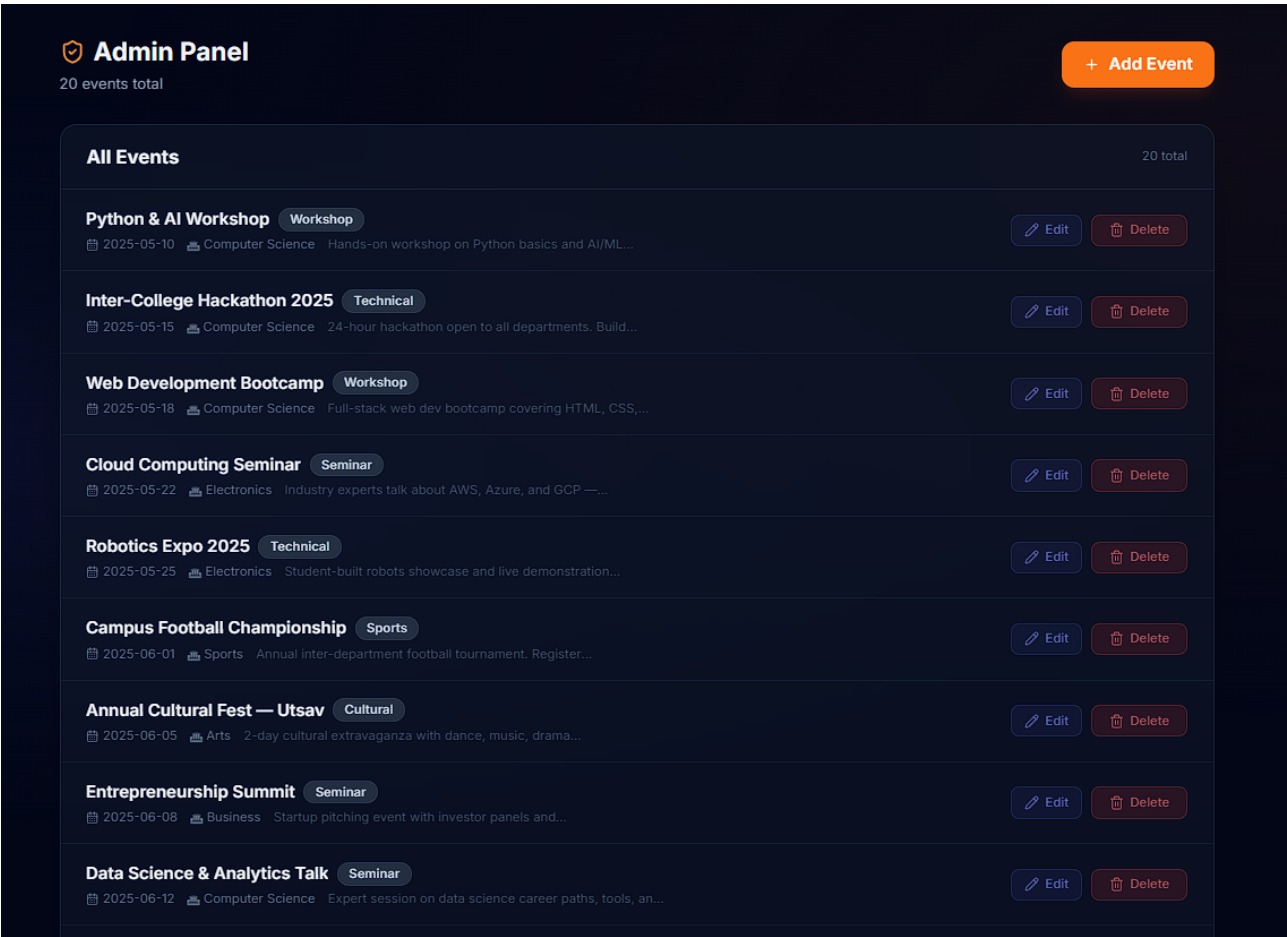


Figure 5.5: Admin Panel - Event Management and Analytics

Figure 5.5 presents the comprehensive admin event list, allowing for rapid management of campus activities with built-in edit and delete functionalities.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The Smart Campus Event & Registration Portal successfully demonstrates a complete, production-ready full-stack web application built using modern technologies. By combining a React SPA frontend with a Spring Boot REST API backend and an H2 relational database, the project delivers a cohesive solution to the real-world problem of campus event management fragmentation.

It simplifies event coordination, ensures secure access via OTP, and provides a polished user experience.

6.2 Future Enhancements

- **Spring Security with JWT:** Integrate Spring Security to issue signed JWT access tokens on login.
- **BCrypt Password Hashing:** Replace plain-text password storage with BCrypt hashing.
- **Redis-Backed OTP Store:** Migrate from in-memory OTP storage to a Redis store to support multi-instance deployment.
- **QR Code Attendance:** Generate unique QR codes per registration for fast check-ins.
- **Push Notifications:** Integrate Firebase Cloud Messaging (FCM) to deliver push notifications.

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

7.1 Mapping of the Project to Complex Engineering Problem (CEP)

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of knowledge	Requires in depth engineering knowledge	WK3, WK4, WK5, WK6	Requires knowledge in Spring Boot, MVC architecture, database design, REST APIs, and frontend technologies (Thymeleaf, HTML/CSS) for developing a scalable event management system.
WP2	Conflicting requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and security by ensuring easy student access while maintaining secure admin authentication and data validation.
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Involves system design, database schema creation, validation handling, and efficient event filtering and registration mechanisms.
WP4	Familiarity	Partially new problems	WK4, WK8	Integrating real-time event updates, user interaction, and admin analytics introduces challenges beyond traditional manual systems.
WP5	Applicable Codes	Limited standard solutions	WK6	Requires custom implementation for event filtering, registration tracking, and statistics generation not fully covered by standard templates.
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Addresses requirements of students, faculty coordinators, and administrators ensuring smooth event organization and participation.
WP7	Interdependence	System level integration	WK3, WK5, WK6	Integrates frontend UI, backend services, database, validation, and security modules into a unified system.

The mapping above highlights how the project addresses complex engineering problems by integrating multiple knowledge areas (WK3-WK8) and balancing conflicting technical requirements such as security versus ease of use.

7.2 Mapping of the Project to Complex Engineering Activities (CEA)

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification based on the Project
EA1	Range of Resources	Use of diverse technologies	Utilizes Spring Boot, Spring MVC, Spring Data JPA, Thymeleaf, HTML/CSS, and relational databases for full-stack development.
EA2	Level of Interactions	Complex interactions between modules	Manages interactions between event management, user registration, validation, authentication, and reporting modules.
EA3	Innovation	Application of modern technologies	Implements a web-based centralized platform with filtering, real-time updates, and secure admin operations.
EA4	Consequences to Society and Environment	Impact on users and society	Enhances student engagement and improves institutional efficiency while reducing manual errors and delays.
EA5	Familiarity	Beyond standard practices	Combines web development, security, and data analytics into a unified system, extending beyond traditional event handling methods.

The complex engineering activities associated with this project demonstrate the use of diverse resources and the management of intricate module interactions to deliver a high-impact institutional solution.

7.3 SDG Mapping (CEP Section)

Table 7.3: SDG Mapping for the Project

SDG No.	SDG Title	Relevance to the Project
SDG 4	Quality Education	Facilitates student participation in academic events, workshops, and seminars, enhancing learning beyond the classroom.
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation in educational institutions through a centralized event management system.
SDG 11	Sustainable Cities and Communities	Supports smart campus initiatives by digitizing event processes and improving resource management.
SDG 16	Peace, Justice and Strong Institutions	Ensures secure and transparent event handling through authentication and controlled access.
SDG 12	Responsible Consumption and Production	Reduces paper usage by enabling online event registration and management.

By aligning with these Sustainable Development Goals, the project contributes to the broader objective of fostering innovation and quality education through digital transformation and responsible resource management.

References

1. For detailed information on backend configuration, refer to the **Spring Boot Documentation** by Pivotal Software Inc. (2024), available at <https://docs.spring.io/spring-boot/docs/current/reference/html/>.
2. The frontend development and component logic follow the official **React v18 Documentation** by Meta Platforms Inc. (2024), found at <https://react.dev/>.
3. Modern build tooling and development server configurations are based on the **Vite: Next Generation Frontend Tooling** guide by Evan You et al. (2024), available at <https://vitejs.dev/guide/>.
4. Styling and responsive UI design principles were implemented using the **Tailwind CSS Documentation** by Tailwind Labs Inc. (2024), available at <https://tailwindcss.com/docs>.
5. The embedded database configuration and management are documented in the **H2 Database Engine** guide by the H2 Group (2024), available at <https://www.h2database.com/>.
6. Data persistence and relational mapping strategies utilize the **Spring Data JPA Reference** guide by VMware Inc. (2024), accessible at <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>.
7. Integration of communication services was performed using the **Twilio WhatsApp API Reference** by Twilio Inc. (2024), available at <https://www.twilio.com/docs/whatsapp>.
8. Frontend-backend communication and API request handling use the **Axios: Promise-based HTTP client** documentation (2024), available at <https://axios-http.com/docs/intro>.
9. Backend testing strategies and unit test implementations are based on the **JUnit 5 User Guide** by the JUnit Team (2024), found at <https://junit.org/junit5/docs/current/user-guide/>.
10. API development and integration testing were validated using the **Postman API Platform Documentation** by Postman Inc. (2024), available at <https://learning.postman.com/docs/>.
11. Architectural principles for web services were guided by the **RESTful Web Services Design Principles** by Lokesh Gupta (2024), available at <https://restfulapi.net/>.
12. The fundamental architecture of the project is inspired by **Fielding, R. T.** (2000), *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. Dissertation, University

of California, Irvine.